

# Security Audit

## Line Next (Smart Wallet)

# Table of Contents

|                                      |    |
|--------------------------------------|----|
| 1. Executive Summary                 | 4  |
| 2. Project Context                   | 4  |
| 3. Audit Scope                       | 7  |
| 4. Security Rating                   | 8  |
| 5. Intended Smart Contract Functions | 9  |
| 6. Code Quality                      | 10 |
| 7. Audit Resources                   | 10 |
| 8. Dependencies                      | 10 |
| 9. Severity Definitions              | 11 |
| 10. Status Definitions               | 12 |
| 11. Audit Findings                   | 13 |
| 12. Centralisation                   | 54 |
| 13. Conclusion                       | 55 |
| 14. Our Methodology                  | 56 |
| 15. Disclaimers                      | 58 |
| 16. About Hashlock                   | 59 |

## CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

## Executive Summary

The Line Next team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

## Project Context

Line Next is an EIP-7702 smart-account (account abstraction) system that lets an EOA delegate execution to a recovery-enabled implementation. Users register a WebAuthn / passkey credential (secp256r1) against their account, and can recover control of the account's assets through a combination of WebAuthn signature verification and a guardian multisig (social recovery).

In short, the account owner is not dependent on a single private key: if the EOA key is lost, a quorum of guardians plus the registered passkey can authorize recovery transactions, while a separate governance contract manages the guardian set, quorum, and executor.

**Project Name:** Line Next

**Project Type:** Smart Wallet

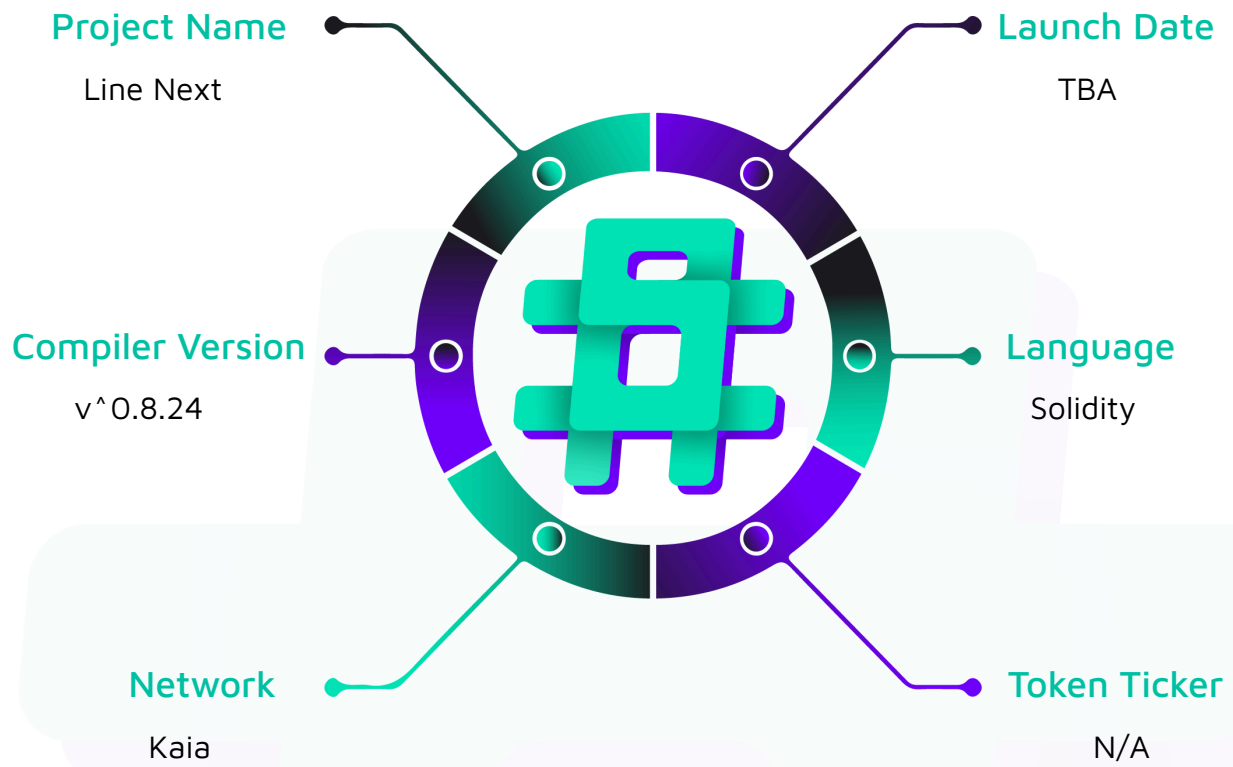
**Compiler Version:** ^0.8.24

**Website:** [www.linenextcorp.com](http://www.linenextcorp.com)

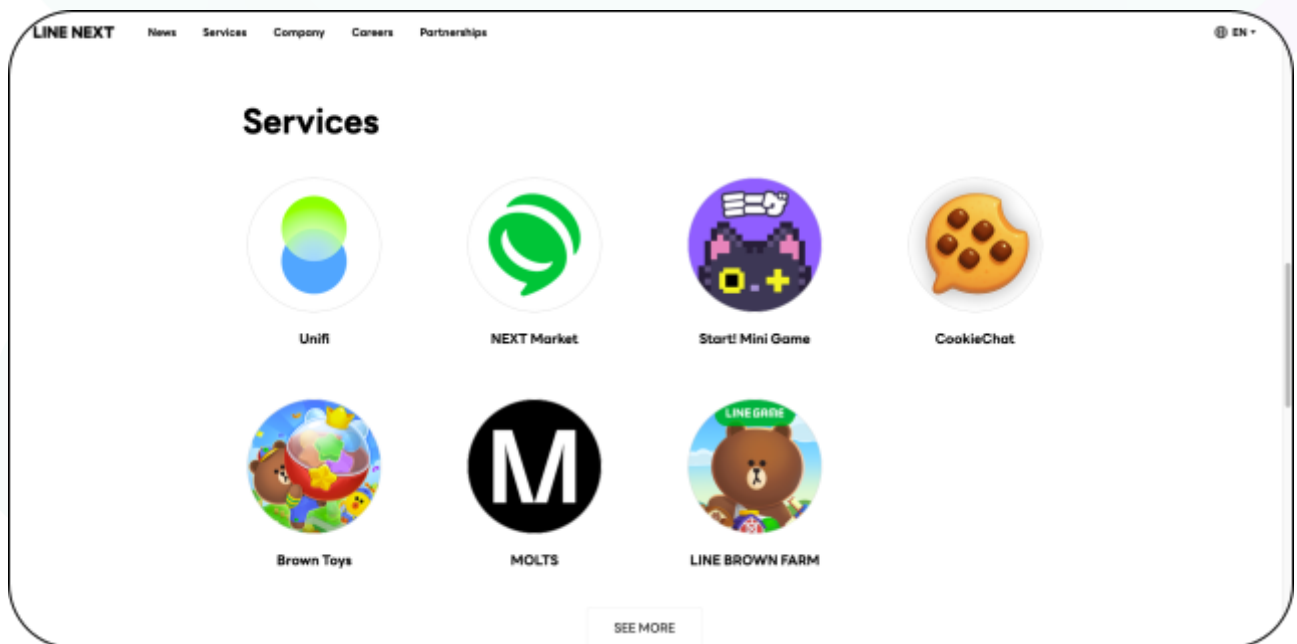
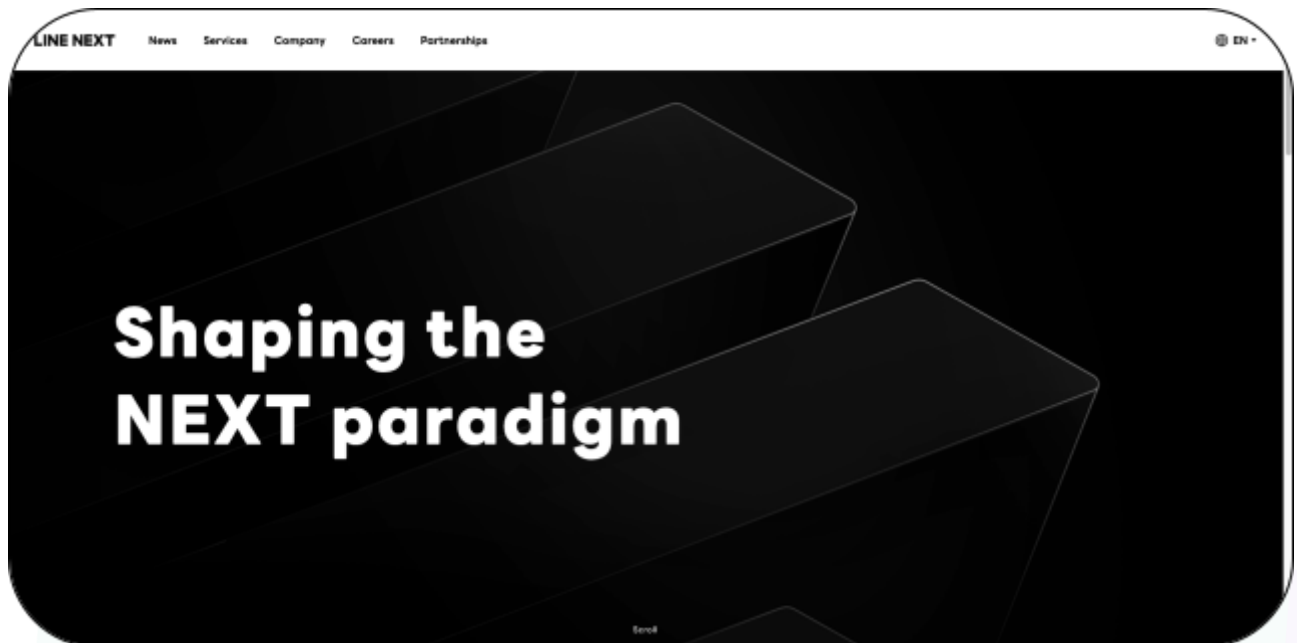
**Logo:**

# LINE NEXT



**Visualised Context:**

## Project Visuals:



## Audit Scope

We at Hashlock audited the solidity code within the Line Next project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual Line Next-by-Line Next analysis and were supported by software-assisted testing.

| Description                   | Line Next Smart Contracts                |
|-------------------------------|------------------------------------------|
| Network                       | Kaia                                     |
| Language                      | Solidity                                 |
| Audit Date                    | June, 2026                               |
| Contract 1                    | Delegate7702Recovery.sol                 |
| Contract 2                    | Delegate7702Verifier.sol                 |
| Contract 3                    | IDelegate7702Verifier.sol                |
| Audited GitHub Commit Hash    | c2bcbac93d48961412d863be9009c24929167ae8 |
| Fix Review GitHub Commit Hash | 9fbe6a1e106e7ac76c0885ca9508e4bdf562330f |

**Note:** Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.

## Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



*The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.*

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

### Hashlock found:

2 High severity vulnerabilities

5 Medium severity vulnerabilities

7 Low severity vulnerabilities

8 QA

**Caution:** Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

## Intended Smart Contract Functions

| Claimed Behaviour                                                                                                                                                                                                                                                                                                                                                                                                                               | Actual Behaviour                      |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| <b>Delegate7702Recovery.sol</b><br>EIP-7702 delegate implementation. Allows the delegating EOA / registered owner to execute batch calls; the executor + one guardian to initialise the WebAuthn (passkey) key on first delegation; the owner + one guardian to update the WebAuthn key; and a guardian + a valid WebAuthn proof to recover the account by executing batch calls. Supports receiving ETH and ERC-721 / ERC-1155 / KIP-37 tokens | Contract achieves this functionality. |
| <b>Delegate7702Verifier.sol</b><br>Guardian multisig governance for the recovery system. Allows guardians to propose and approve adding/removing guardians, changing the quorum, and changing the executor, through a full proposal lifecycle (create / approve / auto-execute at quorum / cancel / expire). Exposes isGuardian and isExecutor consumed by Delegate7702Re                                                                       | Contract achieves this functionality. |
| <b>IDelegate7702Verifier.sol</b><br>Interface for Guardian multisig governance has external isGuardian and isExecutor                                                                                                                                                                                                                                                                                                                           | Contract achieves this functionality. |

## Code Quality

This audit scope involves the smart contracts of the Line Next project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

## Audit Resources

We were given the Line Next project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

## Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

## Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

| Significance | Description                                                                                                                                                                                                          |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| High         | High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.                |
| Medium       | Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions. |
| Low          | Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.                                                                                                       |
| Gas          | Gas Optimisations, issues, and inefficiencies.                                                                                                                                                                       |
| QA           | Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.                                           |



## Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

| Significance        | Description                                                                                                                                                                                                                                                                                                          |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Resolved</b>     | The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.                                                                                   |
| <b>Acknowledged</b> | The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception. |
| <b>Unresolved</b>   | The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.                                                                                                                                                                                               |

# Audit Findings

## High

**[H-01] Delegate7702Recovery#initWebAuthnKey** - first-time initialization sets the account owner and passkey with no EOA consent, enabling account seizure, asset drain, and permanent logout

### Description

On the first call to `initWebAuthnKey` (`s.initialized == false`), the owner consent block is skipped, so the account owner (`s.owner`) and the WebAuthn passkey are written from caller-supplied arguments that are never signed by, nor bound to, the delegating EOA. The function is gated only by the executor (`msg.sender`) plus one guardian signature, below the protocol's own `MIN_QUORUM_FOR_SECURITY = 2`. Because the EOA itself cannot call this function (it is executor-gated) and `owner_` is taken verbatim from the calldata, whoever submits the first init decides who owns the account.

### Vulnerability Details

The function establishes the account's entire root of trust but requires no proof from the account holder on first init:

```
function initWebAuthnKey(
    bytes32 webPX_, bytes32 webPY_, bytes32 rpIdHash_, bytes32 credentialId_,
    uint256 deadline, bytes calldata guardianSig, bytes calldata ownerSig,
    address guardian_, address owner_
) external whenDelegated {
    if (!_isExecutor(msg.sender)) revert InvalidExecutor(); // only executor may call
    if (block.timestamp > deadline) revert DeadlineExpired();
    if (!_isGuardian(guardian_) == false) revert InvalidGuardian();
    // NOTE: no `owner_ == address(0)` check, no proof the EOA authorized `owner_`

    Storage7702Recovery.Layout storage s = Storage7702Recovery.layout();
```

```

bytes32 digest = _hashTypedDataV4(_hashWebAuthnKeyInit(
    webPX_, webPY_, rpIdHash_, credentialId_, s.nonce, deadline, guardian_,
owner_
));

address recoveredGuardian = digest.recover(guardianSig);
if (recoveredGuardian != guardian_) revert InvalidGuardianSignature();

// Owner consent is required ONLY on re-init, skipped entirely on first init:
if (s.initialized) {
    if (owner_ != s.owner) revert OwnerImmutable();
    if (digest.recover(ownerSig) != s.owner) revert InvalidOwnerSignature();
}

s.nonce += 1;
s.webPX = webPX_; s.webPY = webPY_; s.rpIdHash = rpIdHash_; s.credentialId =
credentialId_;
s.owner = owner_; // <-- attacker-chosen owner, never authorized by the
EOA
s.initialized = true;
}

```

## Proof of Concept

```

function test_FirstInit_NoEOAConsent_SeizesAccount() public {
    // `victim` EOA has 7702-delegated to Delegate7702Recovery but never initialized.
    assertEq(Delegate7702Recovery(payable(victim)).isInitialized(), false);

    uint256 deadline = block.timestamp + 1 hours;
    // One guardian G signs the init digest with owner_ = attacker; ownerSig is
empty.
    bytes32 digest = _initDigest(attackerPX, attackerPY, rpIdHash, credId, 0,
deadline, G, attacker);
    bytes memory guardianSig = _sign(gPrivKey, digest);

    vm.prank(executor); // executor relays
    Delegate7702Recovery(payable(victim)).initWebAuthnKey(

```

```

    attackerPX, attackerPY, rpIdHash, credId, deadline,
    guardianSig, /*ownerSig*/ "", G, /*owner_*/ attacker
);

// Attacker is now owner; drains via the owner branch of executeBatchCalls.
Delegate7702Recovery.Call[] memory c = new Delegate7702Recovery.Call[](1);
c[0] = Delegate7702Recovery.Call({to: attacker, value: victim.balance, data:
""});
vm.prank(attacker);
Delegate7702Recovery(payable(victim)).executeBatchCalls(c);

// Victim's later honest init reverts OwnerImmutable -> drained AND locked out.
}

```

## Impact

An executor acting with one guardian gains full control of the victim's account: they set themselves as owner (or plant their own passkey) and can then drain all ETH / ERC20 / ERC721 / ERC1155 via `executeBatchCalls` or the permissionless `recovery()`. The victim is simultaneously locked out, their later honest `initWebAuthnKey` reverts `OwnerImmutable` and `updateWebAuthnKey` reverts `NotOwner`, and the seizure survives EIP-7702 re-delegation. Because the executor and guardians are fleet-wide (single shared `Delegate7702Verifier`), every freshly delegated, not-yet-initialized account is exposed.

## Recommendation

Bind the owner to the delegating EOA, or require the EOA's signature on first init.

## Status

Resolved

## [H-02] Delegate7702Recovery#isValidSignature - cross-chain ERC-1271 replay via EIP-7702 delegation (both branches)

### Description

Under EIP-7702 the same EOA can delegate to Delegate7702Recovery on multiple chains at the same address. Neither branch of isValidSignature binds the challenge to chainId or verifyingContract, so a single off-chain signature valid against the account on chain A is byte-identical to one the same account will accept on chain B. Any ERC-1271 consuming protocol deployed on both chains (Permit2, Seaport, SIWE, arbitrary off-chain order books) can be replayed cross-chain against the same Unifi account.

### Vulnerability Details

Both branches operate on the raw 32-byte hash with no chain- or contract-scoping:

```
// Owner-ECDSA branch (Delegate7702Recovery.sol:434-437)
(address recovered, ECDSA.RecoverError err,) = ECDSA.tryRecover(hash, signature);
if (err == ECDSA.RecoverError.NoError && recovered == address(this)) {
    return ERC1271_MAGIC_VALUE;
}
// Passkey branch (Delegate7702Recovery.sol:440-444) -> verifyWebAuthnSignature
(Recovery:452-480)
// WebAuthn.verify(abi.encodePacked(challenge), ...) // Recovery:468, no chainId, no
address(this)
```

Owner-ECDSA branch: the check is recovered == address(this). Under 7702, address(this) is the EOA, identical across every chain the EOA has delegated on. ECDSA.tryRecover(H, sig) is chain-independent. A legitimate signature the EOA produced for chain A is accepted verbatim on chain B.

Passkey branch: verifyWebAuthnSignature at Recovery:452-480 forwards the raw 32-byte challenge to WebAuthn.verify at line 468. Nothing in the pipeline embeds block.chainid or address(this).

## Proof of Concept

1. Alice's EOA is delegated to `Delegated7702Recovery` on both Ethereum L1 and Base.
2. Alice signs a raw ERC-1271 challenge `H` for a SIWE session on chain L1 (or any consumer that hashes without `chainId`).
3. Attacker relays the same `(H, sig)` to the same consumer on Base against Alice's same-address account.
4. Base consumer calls `account.isValidSignature(H, sig)` -> EOA branch matches -> magic value returned -> authenticated. The identical replay works on the passkey branch when Alice has registered the same passkey on both chains.

Permit2's own EIP-712 embeds `chainId`, so a well-formed Permit2 digest is inadvertently chain-scoped, but any consumer that doesn't do this (SIWE without `chainId`, custom order books, Seaport listings whose message doesn't include `chainId`) is directly exploitable.

## Impact

For every 7702-delegated Unifi account with the same delegation active on more than one chain, any off-chain signature the user produces on one chain is a valid signature against the same account on every other chain. Blast radius scales with the number of chains and the number of ERC-1271 consumers the user interacts with. Neither compromise of the passkey nor of the EOA is required; the user producing a legitimate signature is enough.

## Recommendation

Bind every ERC-1271 signature to its `(chainId, verifyingContract)` tuple before verification. Do not pass the raw caller-supplied hash to either the ECDSA branch or `WebAuthn.verify`; wrap it first with a purpose type-tag, `block.chainid`, and `address(this)`.

## Status

Resolved

## Medium

**[M-01] Delegate7702Verifier#\_addGuardian** - missing `MIN_QUORUM_FOR_SECURITY` re-check enables a single-guardian fleet takeover (quorum-1 deployment)

### Description

`_addGuardian` never re-enforces the `MIN_QUORUM_FOR_SECURITY = 2` floor that the constructor and `_changeQuorum` enforce. From a (1 guardian, quorum 1) deployment, the sole guardian can grow the guardian set while keeping `quorum == 1`, making the multisig look like an N-of-M while remaining a one-key dictatorship. That single key can then swap the executor unilaterally and, because the executor gates `initWebAuthnKey` on every EOA delegated to the shared Recovery implementation, drive H-01 across the entire fleet.

### Vulnerability Details

`_addGuardian` grows `guardians.length` and bumps `policyVersion` but never re-checks the quorum floor that its sibling mutators enforce:

```
// _addGuardian (Delegate7702Verifier.sol:479-489)
function _addGuardian(address newGuardian) internal {
    require(newGuardian != address(0), "Invalid guardian address");
    require(!_isGuardian[newGuardian], "Already a guardian");

    _isGuardian[newGuardian] = true;
    guardians.push(newGuardian);           // length grows...

    emit GuardianAdded(newGuardian);
    unchecked { ++policyVersion; }         // ...but quorum is never re-validated
    emit PolicyVersionBumped(policyVersion, ActionType.ADD_GUARDIAN);
}

// Compare _changeQuorum (Delegate7702Verifier.sol:526-539), which DOES enforce the
// floor:
function _changeQuorum(uint256 newQuorum) internal {
```

```

    require(newQuorum > 0 && newQuorum <= guardians.length, "Invalid quorum");
    if (guardians.length >= 2) {
        require(newQuorum >= MIN_QUORUM_FOR_SECURITY, "Minimum 2 guardians required
for security");
    }
    quorum = newQuorum;
    ...
}

```

The constructor permits the (1 guardian, quorum 1) start (the floor at line 157 is skipped for a single guardian), so adding a guardian reaches {2 guardians, quorum 1}, the exact state the team's own `test_ChangeQuorum_RevertsWhenTwoGuardiansAndQuorumOne` and NatSpec forbid. The executor swap that completes the chain is not even rate-limited: `_changeExecutor` does not bump `policyVersion`.

```

(1 guardian A, quorum 1)
-> A: proposeAddGuardian(B); approveProposal      => {A,B}, quorum 1  [missing floor
re-check]
    -> A: proposeChangeExecutor(attacker); approveProposal (1 == quorum)  => executor
= attacker
    -> attacker (executor) + A (guardian) run the first-init seizure against every
freshly-delegated EOA  [fleet-wide]

```

## Impact

In a (1 guardian, quorum 1) deployment, one guardian key can inflate the guardian set into a fake N-of-M (quorum stays 1), swap in an attacker-controlled executor, and then, together with that executor, seize, drain, and permanently lock out every account delegated to the shared Recovery implementation. Outside a quorum-1 deployment the `_addGuardian` gap still lets the set reach {>=2 guardians, quorum 1}, where one guardian unilaterally controls all governance (add/remove guardians, change quorum, change executor) while newly-added guardians believe in a shared control they do not have.

## Recommendation

After the push in `_addGuardian`:

```

if (guardians.length >= 2 && quorum < MIN_QUORUM_FOR_SECURITY) {

```



```
    quorum = MIN_QUORUM_FOR_SECURITY;    // auto-raise; or revert until quorum is  
    raised first  
}
```

## Status

Resolved



## [M-02] Delegate7702Recovery - missing ERC-1271 (isValidSignature) breaks every off-chain signature workflow the moment a user delegates

### Description

After an EIP-7702 authorization installs Delegate7702Recovery as the EOA's code, the user's address has `extcodesize > 0` and every off-chain signature consumer (Permit2, Seaport/OpenSea, CowSwap / 1inch Fusion, SIWE, Safe co-signing, paymaster / meta-tx relayers, etc.) switches from `ecrecover` to `IERC1271.isValidSignature(bytes32,bytes)`. Delegate7702Recovery does not implement `isValidSignature` and does not advertise the interface in `supportsInterface`, so every such call returns no-data / non-magic and is rejected. The user's canonical signer in this design is a P-256 / WebAuthn passkey that `ecrecover` cannot validate at all, so there is no fallback: the moment the user delegates, the address can no longer produce a signature any off-chain protocol will honour.

### Vulnerability Details

There is no `isValidSignature(bytes32,bytes)` implementation anywhere in the delegate, and `supportsInterface` does not list `0x1626ba7e`:

```
// Delegate7702Recovery.sol:541-548, supportsInterface advertises receivers + ERC165
only
function supportsInterface(bytes4 interfaceId) external pure override returns (bool)
{
    return
        interfaceId == type(IERC721Receiver).interfaceId ||
        interfaceId == type(IERC1155Receiver).interfaceId ||
        interfaceId == 0x01ffc9a7 || // ERC165
        interfaceId == 0x7cc2d017 || // KIP-37 receiver
        interfaceId == type(ERC165).interfaceId;
    // no 0x1626ba7e (IERC1271), off-chain protocols probe this and bail
}
// no `function isValidSignature(bytes32 hash, bytes calldata sig) external view
returns (bytes4)` declared
```

Mechanism, end-to-end:

1. The EOA signs a 7702 authorization -> code at the EOA points to

`Delegate7702Recovery`.

2. The user signs a Permit2 / Seaport / SIWE / CowSwap / Safe payload off-chain. In this protocol's threat model the "real" signer is the registered passkey (P-256), not the original secp256k1 key.

3. The integrator sees `extcodesize(user) > 0` and calls `IERC1271(user).isValidSignature(hash, sig)`, the standard contract-wallet branch.

4. The call hits the delegate's `fallback() external payable {}` (`Delegate7702Recovery.sol:463`), which returns no data -> the integrator does not see the `0x1626ba7e` magic value -> signature rejected. (Even if the user attempted to forward a raw P-256 / WebAuthn blob, there is no on-chain code path that can decode and verify it for an external caller.)

5. There is no alternative: `ecrecover` over the original EOA key is not the trust root in this design (the canonical signer is the passkey), and the passkey is P-256, which `ecrecover` cannot verify.

The break is silent: nothing reverts on delegation; the address just stops being accepted by every signature-consuming protocol.

## Impact

Every off-chain signature workflow stops working for the user the moment they delegate, with no per-dApp workaround (those protocols cannot be patched per user, the fix has to live on the delegate). Even though the user has a working signer (the passkey), there is no on-chain bridge. ERC-1271 is that bridge from "passkey signed this hash" to "address X authorizes hash H", so the passkey is useless off-chain. This nullifies the main UX promise of migrating to a passkey account: the passkey can sign on-chain calls through this contract but cannot sign anything off-chain that other protocols accept. Users discover this only when a swap / listing / login silently fails. No direct fund loss and on-chain control via the passkey -> `executeBatchCalls` path is preserved, but the account becomes economically unusable for the entire signature-based DeFi / NFT / auth stack until the delegate is upgraded.

## Recommendation

Implement ERC-1271 on the delegate and route verification through the same passkey logic already used for on-chain auth, and advertise the interface in `supportsInterface`.

```
bytes4 private constant _ERC1271_MAGIC = 0x1626ba7e;

function isValidSignature(bytes32 hash, bytes calldata signature)
    external view returns (bytes4)
{
    // decode `signature` as the project's WebAuthn / P-256 blob and verify
    // against the currently registered (s.webPX, s.webPY), same path used
    // by recovery(), but read-only and over the externally provided `hash`.
    if (_verifyWebAuthnProof(hash, signature, s.webPX, s.webPY)) {
        return _ERC1271_MAGIC;
    }
    return 0xffffffff;
}

// and advertise 0x1626ba7e in supportsInterface
```

## Status

Resolved

## [M-03] Delegate7702Recovery#isValidSignature - passkey alone can authorize off-chain value transfers via ERC-1271

### Description

The ERC-1271 `isValidSignature` added by the M-02 fix returns the valid-signature magic value for a signature produced by the registered WebAuthn passkey alone, with no guardian. This contradicts the contract's authorization model, in which the passkey is never an autonomous signer; the only on-chain path where the passkey can move value, `recovery()`, also requires a guardian signature over the same digest. The new path lets a passkey-only signer authorize any off-chain, signature-gated action (Permit2 transfers, Seaport orders, SIWE auth), bypassing the passkey+guardian gate.

### Vulnerability Details

On-chain the contract uses three credentials at deliberately different power levels: the EOA key alone controls `executeBatchCalls` (Recovery:345-348), while the passkey can act only when paired with a guardian. `recovery()` requires `_isGuardian(guardian_)` and a guardian signature over the digest (Recovery:345, 362-365). There is no on-chain path where the passkey acts alone.

`isValidSignature` breaks that asymmetry. Its second branch returns the magic value whenever the registered passkey signed the supplied hash, gated only on the public `credentialId/rpIdHash` and the P-256 signature, no guardian, no EOA key, no nonce, no purpose binding:

```
// isValidSignature (Delegate7702Recovery.sol:431-447)
function isValidSignature(bytes32 hash, bytes calldata signature) external view
override returns (bytes4) {
    (address recovered, ECDSA.RecoverError err,) = ECDSA.tryRecover(hash, signature);
    if (err == ECDSA.RecoverError.NoError && recovered == address(this)) {
        return ERC1271_MAGIC_VALUE; // branch 1, EOA key (correct,
mirrors executeBatchCalls)
    }
    if (Storage7702Recovery.layout().initialized) {
        try this.verifyWebAuthnSignature(hash, signature) returns (bool ok) {
            if (ok) return ERC1271_MAGIC_VALUE; // branch 2, passkey ALONE (the
```

```

issue)
    } catch {}
}
return ERC1271_INVALID;
}

// verifyWebAuthnSignature (Delegate7702Recovery.sol:452-480)
// checks only credentialId == s.credentialId, rpIdHash == s.rpIdHash, and the
P-256 signature.

```

The `credentialId` and `rpIdHash` the attacker supplies are public (readable via `getWebAuthnInfo()`, Recovery:403-406); the only secret is the passkey itself. Both `verifyWebAuthnSignature` (Recovery:467) and `recovery()`'s `_verifyWebAuthnProof` (Recovery:506) also feed a bare 32-byte value to `WebAuthn.verify` with no domain tag, so a passkey assertion is context-free.

### Proof of Concept

1. The account is set up normally (EOA owner + registered passkey + guardians) and holds the standard max approval to Permit2 (or a Seaport conduit).
2. Attacker compromises only the passkey; the single-factor-loss event the passkey+guardian design exists to survive, with no guardian and no EOA key.
3. Attacker builds a Permit2 `PermitTransferFrom` digest `H` draining the account's USDC to themselves, signs a WebAuthn assertion over `H` with the passkey, and abi.encodes it as a `WebAuthnProof`.
4. Attacker calls Permit2 -> Permit2 sees the account has code -> calls `account.isValidSignature(H, blob)` -> branch 2 verifies -> returns `0x1626ba7e` -> Permit2 transfers the funds. The same works for Seaport orders (NFTs) and any SIWE/ERC-1271 auth.

### Impact

A thief who compromises only the passkey can drain every asset the account has pre-approved to an ERC-1271-consuming protocol (Permit2 allowances, Seaport/conduit approvals). For a typical DeFi user with a max Permit2 approval, most of their fungible tokens, with no guardian and no EOA key. Raw ETH and un-approved tokens are not reachable this way (they still require the EOA key via `executeBatchCalls`



or a guardian via `recovery()`). The passkey+guardian recovery design exists precisely to make this single-factor compromise survivable; the ERC-1271 path defeats that protection for off-chain-approved assets.

## Recommendation

Two paths are acceptable depending on the client's product requirements. The trade-off is explicit: Option A restores the passkey+guardian invariant; Option B ships the passkey as a first-class off-chain signer and accepts the loss surface that comes with it.

Option A (invariant-restoring, preferred): drop the passkey branch. Make `isValidSignature` validate only the owner-ECDSA branch (`recovered == address(this)`, mirroring `executeBatchCalls`) and drop the passkey branch. This restores the on-chain invariant that the passkey never acts alone. Note that the ECDSA branch still needs ERC-7739 (or equivalent `chainId + address(this)` binding).

```
function isValidSignature(bytes32 hash, bytes calldata signature) external view
override returns (bytes4) {
    (address recovered, ECDSA.RecoverError err,) = ECDSA.tryRecover(hash, signature);
    if (err == ECDSA.RecoverError.NoError && recovered == address(this)) return
    ERC1271_MAGIC_VALUE;
    return ERC1271_INVALID;
}
```

Option B: keep the passkey branch as a documented risk acceptance. If the product intentionally treats the passkey as a primary off-chain signer, keep the branch but make the trade-off explicit.

## Status

Resolved

## [M-04] Delegate7702Recovery#isValidSignature - same passkey across multiple 7702-delegated accounts enables cross-account replay

### Description

Delegate7702Recovery binds a WebAuthn credential to storage but does not bind it to the account address. A user who registers the same synced platform passkey (iCloud Keychain, Google Password Manager, 1Password) against two of their own EOAs, a common UX pattern for people with more than one wallet, produces two accounts where the same (hash, signature) blob validates. An ERC-1271 consumer keyed on (verifyingContract, hash) will accept the same signature against either account.

### Vulnerability Details

verifyWebAuthnSignature (Recovery.sol:452-480) checks only:

```
if (proof.credentialId != s.credentialId) return false; // Recovery:456
if (rpIdHash != s.rpIdHash) return false; // Recovery:465
return WebAuthn.verify(abi.encodePacked(challenge), ..., s.webPX, s.webPY); //
Recovery:467
```

Both credentialId and rpIdHash are the same value across every account that registered the same passkey; the P-256 pubkey (webPX, webPY) is identical too. Nothing in the path binds the account's own address. The signature sig produced by the passkey over hash H returns a magic value from account\_A.isValidSignature(H, sig) and account\_B.isValidSignature(H, sig) whenever both accounts share the passkey. The same concern extends to the owner-ECDSA branch (line 434-437) if one EOA is delegated to Delegate7702Recovery via a proxy pattern that maps two 7702-delegated addresses to the same underlying signing key, but the primary realistic vector is passkey sync.

### Proof of Concept

1. Alice owns EOA-A and EOA-B, both delegated to Delegate7702Recovery, both initWebAuthnKey-registered with the same platform passkey (same webPX/webPY/credentialId/rpIdHash).
2. Alice signs a Permit2 digest H intending to authorize EOA-A to spend her USDC; the signature is a WebAuthn assertion over H.



3. Attacker submits the same (H, blob) to Permit2 with `owner = EOA-B`. Permit2 calls `EOA-B.isValidSignature(H, blob)` -> passkey branch matches (same key, same `credentialId`) -> magic value -> USDC leaves EOA-B.

## Impact

Users with multi-wallet setups sharing one passkey have every off-chain-signed action fungible across their own accounts. If those accounts have different approvals (e.g. one holds tokens the user intended to keep isolated), the isolation is defeated. The passkey doesn't need to be compromised; an accidental replay by a filler or an intentional targeting by an attacker who observed a legitimate signature is sufficient.

## Recommendation

It is recommended to bind `address(this)` into the challenge before validation on both branches.

## Status

Resolved

## [M-05] Delegate7702Recovery#verifyWebAuthnSignature - no ERC-1271 context tag on WebAuthn challenge (context confusion)

### Description

Even without any credential compromise and without multi-account setups, the passkey branch treats every 32-byte hash identically. A WebAuthn assertion produced in one context (e.g. "sign in with Ethereum" on the web app) is bitwise the same as an assertion authorizing a Permit2 transfer. A malicious dApp served from the passkey's rpId origin (a compromised RP frontend, an XSS on the real domain, a subdomain takeover, or a supply-chain compromise of a frontend dependency) can request an innocent-looking WebAuthn assertion and present it to a Permit2 filler as spend authorization.

### Vulnerability Details

Recovery.sol:468 `abi.encodePacked(challenge)` passes the raw hash straight into `WebAuthn.verify`. No purpose prefix, no domain separator, no type tag. The same shape appears in `_verifyWebAuthnProof` at line 507. From the browser's perspective, a `navigator.credentials.get({ challenge: H })` call is a single opaque request, it cannot distinguish "H is a login nonce" from "H is a Permit2 digest," and neither can the contract on receipt.

```
// ...
(address recovered, ECDSA.RecoverError err,) = ECDSA.tryRecover(hash, signature);
if (err == ECDSA.RecoverError.NoError && recovered == address(this)) {
    return ERC1271_MAGIC_VALUE;
}

// 2) passkey (WebAuthn) 경로, 초기화된 경우에만. self-staticcall + try/catch 로
// malformed blob 의 abi.decode revert 흡수.
if (Storage7702Recovery.layout().initialized) {
    try this.verifyWebAuthnSignature(hash, signature) returns (bool ok) {
        if (ok) return ERC1271_MAGIC_VALUE;
    } catch {}
}
// ...
```

## Proof of Concept

1. Attacker compromises any origin served under the account's `rpId` (via XSS on a real subdomain, a subdomain takeover of a dangling cloud bucket, or a malicious dependency merged into the RP frontend).
2. Malicious JS running there constructs a Permit2 digest `H` draining the user's USDC and calls `navigator.credentials.get({ challenge: H })` under the guise of "confirm your identity to continue."
3. The user's platform passkey (Touch ID / Face ID / Windows Hello) prompts the user to, assuming it's a routine login, approve. The returned WebAuthn assertion is over `H`.
4. Attacker submits the assertion to Permit2 as a `WebAuthnProof` blob -> `isValidSignature(H, blob)` returns magic value -> funds transferred.

## Impact

Any dApp with legitimate `rpId` access, or any attacker who briefly captures such access (subdomain takeover, XSS on any origin under the registered `rpId` scope, supply-chain compromise of a frontend dependency, malicious code injected into a real RP frontend), can farm passkey signatures that are indistinguishable from spend authorizations. The attack does not require the passkey's private key to leave the authenticator device; a single tapped biometric under a misleading prompt (e.g. "confirm your identity to continue") is enough for the attacker to obtain a WebAuthn assertion over any hash they choose, including a Permit2 digest that drains the account's pre-approved tokens.

## Recommendation

Prefix the challenge with an ERC-1271-specific domain tag before hashing, e.g.

```
bytes32 boundChallenge = keccak256(
    abi.encode(keccak256("UNIFI_ERC1271_v1"), block.chainid, address(this), hash)
);
// then pass boundChallenge (not hash) to WebAuthn.verify
```

## Status

Resolved

## Low

**[L-01] Delegate7702Recovery#initWebAuthnKey** - owner\_ not validated against the zero address

### Description

First init accepts `owner_ == address(0)` and writes it to `s.owner`, permanently disabling every owner-gated path.

### Vulnerability Details

There is no zero-check on `owner_` (flagged inline at line 127). With `s.owner == address(0)`:

```
// updateWebAuthnKey (Delegate7702Recovery.sol:258)
if (msg.sender != s.owner) revert NotOwner();           // msg.sender can never be
address(0), always reverts

// executeBatchCalls (Delegate7702Recovery.sol:345-348)
if (msg.sender != address(this)) {
    if (!s.initialized) revert NotInitialized();
    if (msg.sender != s.owner) revert NotOwner();       // registered-owner branch is
dead
}
```

`updateWebAuthnKey` becomes uncallable and the registered-owner branch of `executeBatchCalls` is dead. (The EOA-self branch of `executeBatchCalls` and `recovery()` still work, so the account is not fully bricked.)

### Impact

`updateWebAuthnKey` becomes uncallable and the registered-owner branch of `executeBatchCalls` is permanently dead, so passkey rotation and owner-initiated batch calls are disabled for that account for the rest of its lifetime. No fund loss; the EOA-self batch path and `recovery()` are unaffected.

## Recommendation

Add `if (owner_ == address(0)) revert InvalidOwner();` on the init path, or bind `owner_ == address(this)` on first init (either one closes it).

## Status

Resolved



**[L-02] Delegate7702Recovery#updateWebAuthnKey** - rotates the passkey with no proof of the current passkey

## Description

updateWebAuthnKey overwrites the registered passkey without verifying possession of the existing one; the only authorisation is the owner key plus a single guardian co-signature.

## Vulnerability Details

```
// updateWebAuthnKey (Delegate7702Recovery.sol:255-282)
if (!s.initialized) revert NotInitialized();
if (msg.sender != s.owner) revert NotOwner();           // owner key
...
address recoveredGuardian = digest.recover(guardianSig);
if (recoveredGuardian != guardian_) revert InvalidGuardianSignature(); // + one
guardian
...
s.webPX = webPX_;
s.webPY = webPY_;           // new passkey written, no current-passkey proof
```

No WebAuthn proof over the current (webPX, webPY) is required to replace it.

## Impact

The passkey is the factor recovery() relies on to survive an owner-key compromise. An attacker who has already compromised the owner key can, with one guardian's routine-rotation co-sign, wipe the legitimate passkey and install their own, neutralizing the recovery factor that exists to survive exactly that compromise, so the legitimate user can no longer recover and the attacker holds the only registered passkey.

## Recommendation

Require a valid WebAuthn proof over the currently-registered passkey on rotation (prove possession of the old key before replacing it), or route passkey changes through the quorum-gated recovery flow.

**Status**

Resolved



## [L-03] Delegate7702Recovery#initWebAuthnKey / updateWebAuthnKey - WebAuthn key coordinates not validated

### Description

(webPX\_, webPY\_) are stored with no on-curve / non-zero validation; a malformed key is accepted and only rejected later at verification time, bricking recovery.

### Vulnerability Details

```
// initWebAuthnKey (Delegate7702Recovery.sol:156-157), identical in
updateWebAuthnKey:279-280
s.webPX = webPX_;
s.webPY = webPY_;    // no check that (webPX_, webPY_) is a valid, non-zero P-256
point
```

OZ P256.verify returns false for an off-curve or zero public key, so the stored key is simply never usable, but the failure surfaces only when recovery() is attempted, not at registration.

### Impact

A malformed key is accepted at registration but silently disables the recovery path for that account. recovery() reverts at P-256 verification time because the stored key is unusable, so the account loses its recovery capability. No forgery or signature bypass (P-256 rejects invalid keys).

### Recommendation

Validate the coordinates at registration (non-zero, on-curve) and revert on an invalid key, so the failure is caught at init rather than at recovery time.

### Status

Resolved



## [L-04] Delegate7702Verifier#\_createProposal - no maximum proposal duration lets a far-future proposal lock the dedup-hash slot

### Description

`_createProposal` enforces a minimum duration but no maximum, so a guardian can set a near-infinite deadline. Because re-proposing the same (action, target) is blocked by the active-proposal hash, and both public relief paths require expiry, only the proposer (or an unrelated policy change) can clear it.

### Vulnerability Details

```
// _createProposal (Delegate7702Verifier.sol:424)
require(duration >= MIN_PROPOSAL_DURATION, "Duration too short"); // no upper bound
...
proposal.deadline = block.timestamp + duration;
```

### Impact

A guardian can file a far-future, never-approved `REMOVE_GUARDIAN(accomplice)` proposal to occupy that dedup slot, blocking anyone else from proposing the accomplice's removal. The block persists until the proposer voluntarily cancels, or until an unrelated `ADD/REMOVE/CHANGE_QUORUM` execution bumps `policyVersion` (after which the stale blocker becomes publicly cancellable). Each proposer can occupy at most `MAX_ACTIVE_PER_PROPOSER` (5) such slots and cannot starve other proposers' actions.

### Recommendation

Enforce a `MAX_PROPOSAL_DURATION` ceiling in `_createProposal` so the "anyone after deadline" relief always becomes reachable in bounded time.

### Status

Resolved

## [L-05] Delegate7702Verifier#\_removeGuardian - unbounded guardian set raises removal gas (no MAX\_GUARDIANS)

### Description

`guardians[]` has no upper bound; `_removeGuardian` iterates the active-proposal set ( $\leq \text{guardians.length} \times \text{MAX\_ACTIVE\_PER\_PROPOSER}$ ) and the guardian array, so a very large set raises governance-op cost and can eventually make removal unaffordable.

### Vulnerability Details

```
// _removeGuardian (Delegate7702Verifier.sol:501-519)
uint256 n = _activeProposalIds.length;           // <= guardians.length *
MAX_ACTIVE_PER_PROPOSER (5)
for (uint256 i = 0; i < n; ) { ... if (p.approvals[guardianToRemove]) { ... } ... }
// approval stripping
for (uint256 i = 0; i < guardians.length; ) { ... } // linear scan + swap-and-pop
```

There is no `MAX_GUARDIANS` cap in the constructor or `_addGuardian`.

### Impact

At a large guardian count, `_removeGuardian` (which loops the active-proposal set and the guardian array) and `cleanupExpiredProposals` (batch-capped at 50) become progressively more gas-expensive, and guardian removal could eventually exceed the block gas limit and become impossible. Governance is quorum-gated, so the set is grown only through legitimate (but unbounded) additions, not spam.

### Recommendation

Introduce a sane `MAX_GUARDIANS` upper bound enforced in `_addGuardian` and the constructor.

### Status

Resolved

**[L-06] Delegate7702Verifier** - nothing prevents executor == guardians, collapsing the first-init two-party gate

## Description

Neither the constructor nor `_changeExecutor` forbids the executor address from also being a guardian, so a single entity can satisfy both factors of the first-init gate alone.

## Vulnerability Details

```
// constructor (Delegate7702Verifier.sol:142, 144-153), executor and guardians set
independently, no overlap check
executor = initialExecutor;
for (...) { ... _isGuardian[guardian] = true; guardians.push(guardian); ... }

// _changeExecutor (Delegate7702Verifier.sol:541-545)
function _changeExecutor(address newExecutor) internal {
    require(newExecutor != address(0), "Invalid executor address");
    executor = newExecutor;                // no check that newExecutor is not
a guardian
    ...
}
```

The first-init path (`initWebAuthnKey`) is gated on `_isExecutor(msg.sender)` + one guardian signature; if those are the same key, the "two independent parties" assumption is void.

## Impact

One key that is simultaneously the executor and a guardian satisfies both factors of the first-init gate by itself, so a single key, with no second, independent party, can establish the owner/passkey on a freshly-delegated account and seize it. The separation-of-duties assumption the gate relies on (executor and guardian being distinct parties) does not hold.

## Recommendation

Reject executor in guardians in the constructor, `_changeExecutor`, and `_addGuardian`.

**Status**

Resolved



## [L-07] Delegate7702Verifier#\_removeGuardian - guardian set can shrink below the constructor's MIN\_GUARDIANS

### Description

The M-01 fix added a `MIN_GUARDIANS = 3` floor enforced in the constructor, but `_removeGuardian` and `proposeRemoveGuardian` gate only on quorum, never on `MIN_GUARDIANS`. The guardian set can therefore be reduced from 3 to a 2-of-2, below the deploy-time minimum the fix established, leaving governance with no fault tolerance.

### Vulnerability Details

The removal path checks only quorum-related bounds:

```
// proposeRemoveGuardian (Delegate7702Verifier.sol:248)
if (!(guardians.length - 1 >= quorum)) revert CannotReduceBelowQuorum();

// _removeGuardian (Delegate7702Verifier.sol:564-565)
if (!(guardians.length > 1)) revert CannotRemoveLastGuardian();
if (!(guardians.length > quorum)) revert CannotReduceBelowQuorum();
// no MIN_GUARDIANS check, unlike the constructor (Verifier:199)
```

So the invariant is established at deploy and not maintained:

1. Deploy {A,B,C}, quorum 2.
2. Remove C (`proposeRemoveGuardian`: `3-1 >= 2` passes; `_removeGuardian`: `3 > 1`, `3 > 2` pass) -> {A,B}, quorum 2, below the enforced minimum of 3.

The reachable floor is exactly {2 guardians, quorum 2}: further removal needs `2 > quorum`, i.e. `quorum == 1`, but quorum is forced `>= 2` whenever there are `>= 2` guardians (`constructor`:201, `_changeQuorum`:602-604, `_addGuardian`:553-556), so `guardians.length` can never reach 1. The no-single-key / multisig invariant therefore always holds (`guardians >= 2`, `quorum >= 2`); this is not a single-key takeover and M-01 is not reintroduced.

## Impact

Governance can be reduced to a 2-of-2, below the contract's own advertised `MIN_GUARDIANS = 3`. A 2-of-2 has no fault tolerance: if either of the two remaining guardian keys is subsequently lost or unavailable, every governance action permanently deadlocks. No guardian can be added or removed, quorum cannot be changed, and the executor cannot be changed, because none can ever reach 2 approvals again. Because `Delegate7702Recovery.updateWebAuthnKey` requires `verifier.quorum()` guardian signatures (Recovery:298-299), fleet-wide passkey rotation also freezes with no recovery path. No funds are at risk and no takeover is possible; the harm is loss of governance availability.

## Recommendation

Mirror the constructor floor on the removal path: `require(guardians.length - 1 >= MIN_GUARDIANS)` in `proposeRemoveGuardian` and `guardians.length > MIN_GUARDIANS` in `_removeGuardian`, or explicitly document and accept degradation to a 2-of-2.

## Status

Acknowledged

## QA

### [Q-01] Delegate7702Verifier#approveProposal - proposer can approve its own proposal (no proposer/approver separation)

#### Description

approveProposal does not exclude the proposer, so a guardian who proposes can also cast one of the quorum approvals on their own proposal.

#### Vulnerability Details

```
// approveProposal (Delegate7702Verifier.sol:239-256)
function approveProposal(uint256 proposalId) external onlyGuardian {
    ...
    require(!proposal.approvals[msg.sender], "Already approved"); // no `msg.sender
    != proposal.proposer` check
    proposal.approvals[msg.sender] = true;
    proposal.approvalCount += 1;
    ...
}

// _createProposal does NOT pre-credit the proposer (Delegate7702Verifier.sol:431-444
// writes nothing to approvals/approvalCount)
```

The proposer earns no free vote (creation writes nothing to approvals / approvalCount), so reaching requiredQuorum still needs that many distinct guardian approvals, but one of them may be the proposer's own.

#### Impact

The proposer's own approval counts toward quorum, so a malicious quorum can form from the proposer plus Q-1 other guardians rather than Q guardians other than the proposer. This does not weaken the documented quorum invariant: with quorum  $\geq 2$  a single guardian still cannot execute alone, and proposing grants no free vote (creation writes nothing to approvals / approvalCount).

## Recommendation

If strict proposer/approver separation is desired, add `require(msg.sender != proposal.proposer, "Proposer cannot approve own proposal")`. Note this then needs quorum approvers besides the proposer (effectively quorum + 1 participating guardians).

## Status

Acknowledged



## [Q-02] Delegate7702Verifier#getProposalStatus - canExecute is vestigial (always false between transactions)

### Description

`getProposalStatus.canExecute` can never read true across transaction boundaries, because the quorum-reaching approval auto-executes atomically.

### Vulnerability Details

```
// getProposalStatus (Delegate7702Verifier.sol:417-419)
canExecute = isActive && !isExpired && proposal.approvalCount >=
proposal.requiredQuorum;

// approveProposal (Delegate7702Verifier.sol:253-254)
if (proposal.approvalCount >= proposal.requiredQuorum) {
    _executeProposal(proposalId);    // executes in the same tx -> state becomes
EXECUTED
}
```

An ACTIVE proposal can never persist with `approvalCount >= requiredQuorum` (it is EXECUTED the instant it reaches it); a stale proposal can't reach quorum either (approve reverts on the staleness gate at line 245 before incrementing). So any off-chain reader sees `canExecute == false` forever.

### Impact

Misleads integrators into expecting a manual execute step the protocol does not have. No on-chain security impact.

### Recommendation

Remove `canExecute`, or document that execution is automatic at quorum and the field is always false when read between transactions.

### Status

Resolved

## [Q-03] Delegate7702Verifier#proposeChangeQuorum - quorum floor validated at execute, not at propose

### Description

proposeChangeQuorum does not check MIN\_QUORUM\_FOR\_SECURITY; the floor is enforced only at execution, so an invalid quorum = 1 proposal (with >= 2 guardians) can be created and approved and then reverts on the quorum-reaching approval.

### Vulnerability Details

```
// proposeChangeQuorum (Delegate7702Verifier.sol:206-207), no MIN_QUORUM_FOR_SECURITY check
require(newQuorum > 0 && newQuorum <= guardians.length, "Invalid quorum");
require(newQuorum != quorum, "Same as current quorum");

// _changeQuorum (Delegate7702Verifier.sol:531-533), floor enforced here, at execute time
if (guardians.length >= 2) {
    require(newQuorum >= MIN_QUORUM_FOR_SECURITY, "Minimum 2 guardians required for security");
}
```

### Impact

Liveness / griefing: the invalid proposal consumes an active slot and reverts on the final (quorum-reaching) approver instead of being rejected at creation. No security impact.

### Recommendation

Validate the MIN\_QUORUM\_FOR\_SECURITY floor in proposeChangeQuorum so it fails fast at propose time.

### Status

Resolved

## [Q-04] Delegate7702Verifier#\_changeExecutor - does not bump policyVersion (in-flight executor proposals not serialized)

### Description

The three governance actions that affect the approval set bump `policyVersion` to stale in-flight proposals; `_changeExecutor` does not, so competing `CHANGE_EXECUTOR` proposals are last-write-wins and survive interleaved guardian/quorum changes.

### Vulnerability Details

```
// _changeExecutor (Delegate7702Verifier.sol:541-545), no ++policyVersion, unlike the
// other three actions
function _changeExecutor(address newExecutor) internal {
    require(newExecutor != address(0), "Invalid executor address");
    executor = newExecutor;
    emit ExecutorChanged(newExecutor);
}

// cf. _addGuardian:487, _removeGuardian:522, _changeQuorum:537 all do `unchecked {
// ++policyVersion; }`
```

This is intentional (the executor is not part of quorum math, per the `policyVersion` NatSpec), but it means executor-change proposals are not serialized the way the other actions are.

### Impact

Two competing `CHANGE_EXECUTOR` proposals to different targets are not mutually serialized: executing the first does not stale the second (because `_changeExecutor` does not bump `policyVersion`), so both remain executable and the outcome is last-write-wins. The other three governance actions, by contrast, invalidate each other on execution. There is no approval-set bypass; every executor change still requires a full quorum of distinct approvals.

### Recommendation

Consider bumping `policyVersion` on executor change as well (or otherwise serializing `CHANGE_EXECUTOR`), given the executor's elevated role.

**Status**

Acknowledged



## [Q-05] Delegate7702Recovery#\_getRevertMsg - sub-68-byte reverts collapse to a generic string

### Description

`_getRevertMsg` returns "Transaction reverted silently" for any returndata shorter than 68 bytes, so custom errors and `Panic(uint256)` (both < 68 bytes) lose their real reason in recovery / `executeBatchCalls` failures.

### Vulnerability Details

```
// _getRevertMsg (Delegate7702Recovery.sol:442-449)
function _getRevertMsg(bytes memory returnData) internal pure returns (string memory)
{
    if (returnData.length < 68) return "Transaction reverted silently"; // custom
    error (4B) / Panic (36B) lost
    assembly { returnData := add(returnData, 0x04) }
    return abi.decode(returnData, (string)); // assumes
    Error(string) layout
}
```

### Impact

Degraded observability only: the batch still reverts; no fund or state effect. Integrators debugging a failed recovery/batch lose the underlying error.

### Recommendation

Bubble up the raw returndata (e.g. `assembly revert(add(returnData, 0x20), mload(returnData))`) or branch on the 4-byte error selector so custom errors and `Panic` are preserved.

### Status

Resolved

## **[Q-06] Style and gas** - mixed error handling, loop increments, blanket payable fallback

### **Description**

A cluster of code-quality items with no security impact.

### **Vulnerability Details**

- `Delegate7702Verifier` uses `require-strings` throughout; `Delegate7702Recovery` uses custom error types. Inconsistent, and `require-strings` are more expensive.
- The constructor loop uses `checked i++` (`Delegate7702Verifier.sol:144`) while every other loop uses `unchecked { ++i }`.
- `fallback() external payable {}` (`Delegate7702Recovery.sol:463`): under 7702 delegation any unmatched selector succeeds silently instead of reverting, masking caller/integration mistakes (`receive()` at line 457 already handles bare ETH).

### **Impact**

None (gas / readability / robustness).

### **Recommendation**

Standardise on custom errors, align loop increments, and consider removing the catch-all fallback (keep `receive`) so unexpected calls revert loudly.

### **Status**

Resolved

## [Q-07] Delegate7702Verifier#\_createProposal - per-proposer active cap is global, not per-action-type

### Description

The `MAX_ACTIVE_PER_PROPOSER` cap counts all of a proposer's active proposals regardless of `ActionType`, so a proposer holding 5 active proposals of one type cannot open a proposal of a different type until one finalizes.

### Vulnerability Details

```
// _createProposal (Delegate7702Verifier.sol:426-429, 442)
require(
    _activeCountByProposer[msg.sender] < MAX_ACTIVE_PER_PROPOSER,    // counts ALL
    action types together
    "Too many active proposals from sender"
);
...
_activeCountByProposer[msg.sender] += 1;    // incremented for every action type
```

A proposer with 5 active `ADD_GUARDIAN` proposals cannot open a `REMOVE_GUARDIAN` / `CHANGE_QUORUM` / `CHANGE_EXECUTOR` proposal until one finalizes.

### Impact

UX / liveness friction only, and self-inflicted: it limits the proposer's own ability to create more proposals, not anyone else's. Mitigated: the proposer can `cancelProposal` one of their own to free a slot. No security impact (a global cap is in fact stricter than a per-action cap).

### Recommendation

If the friction matters, track the cap per `ActionType`; otherwise document the behavior. No security change required.

### Status

Acknowledged

**[Q-08] Delegate7702Verifier#approveProposal / \_executeProposal** - executing any guardian-set / quorum proposal bumps policyVersion, marking every other in-flight proposal stale

## Description

Every proposal snapshots the current `policyVersion` into `requiredPolicyVersion` at creation time. The three governance actions that affect the approval set (`_addGuardian`, `_removeGuardian`, `_changeQuorum`) each increment `policyVersion` on execution. The moment one of them executes, every other ACTIVE proposal's snapshot no longer matches the global `policyVersion`, so any further `approveProposal` or `_executeProposal` on it reverts "Proposal stale". Those proposals can no longer progress; they must be cancelled and then re-proposed and re-approved from scratch. This is the complement of Q-04 (which notes `_changeExecutor` is the one action that deliberately does not bump `policyVersion`).

## Vulnerability Details

```
// Snapshot at creation, _createProposal, Delegate7702Verifier.sol:438
proposal.requiredPolicyVersion = policyVersion; // 현재 policy 버전 고정 (정책 변경
stale 차단)

// Stale guard in the approval path, approveProposal, Delegate7702Verifier.sol:245
require(proposal.requiredPolicyVersion == policyVersion, "Proposal stale");

// Stale guard in the execute path (defense-in-depth), _executeProposal,
Delegate7702Verifier.sol:455
require(proposal.requiredPolicyVersion == policyVersion, "Proposal stale");

// Each of the three approval-set actions bumps policyVersion on execution:
// _addGuardian (Delegate7702Verifier.sol:487) unchecked { ++policyVersion; }
// @audit "if 4 active addGuardian proposals exist at policyVersion
x1 and one executes,
// policyVersion becomes x2 and the other 3 proposals are
stale."
// _removeGuardian (Delegate7702Verifier.sol:522) unchecked { ++policyVersion; }
// _changeQuorum (Delegate7702Verifier.sol:537) unchecked { ++policyVersion; }
```



Worked example: four `ADD_GUARDIAN` proposals are created while `policyVersion == 1`, so all four carry `requiredPolicyVersion == 1`. Guardians approve the first to quorum; `approveProposal` calls `_executeProposal`, which runs `_addGuardian` and bumps `policyVersion` to 2. The remaining three proposals still hold `requiredPolicyVersion == 1`, so every subsequent `approveProposal(id)` on them fails the L245 stale guard, and even a proposal that was already at quorum would fail the L455 guard if execution were re-attempted. The cleanup path is permissionless: a stale proposal satisfies `proposal.requiredPolicyVersion != policyVersion`, so anyone may clear it via `cancelProposal` (`Delegate7702Verifier.sol:277`), which frees the dedup hash and the proposer's active-slot count; the change must then be re-proposed and re-approved.

Crucially, prior approvals are not carried across the bump: the `requiredPolicyVersion` mismatch invalidates a proposal regardless of how many guardians had already approved it, so those votes are effectively discarded and must be re-cast on the re-proposed item.

## Impact

Liveness / UX only, no authorization bypass. Concurrent governance is serialized to one execution per policy epoch: executing one change strands every other in-flight proposal, discarding any approvals already cast and forcing a cancel + re-propose + re-approval cycle. Griefing potential is minimal (re-proposal is cheap; stale proposals are permissionlessly cancellable).

## Recommendation

Accept as designed (the staleness deliberately prevents a decision approved against an outdated guardian set / quorum from executing against a newer one), but document that governance changes are serialized per policy epoch: approve and execute one change at a time, then re-propose any others. If concurrent batching is genuinely desired, snapshot each proposal against only the sub-state its action depends on (e.g. the quorum value, or specific guardian membership) rather than a single monotonic `policyVersion`, so that independent changes no longer invalidate one another.

**Status**

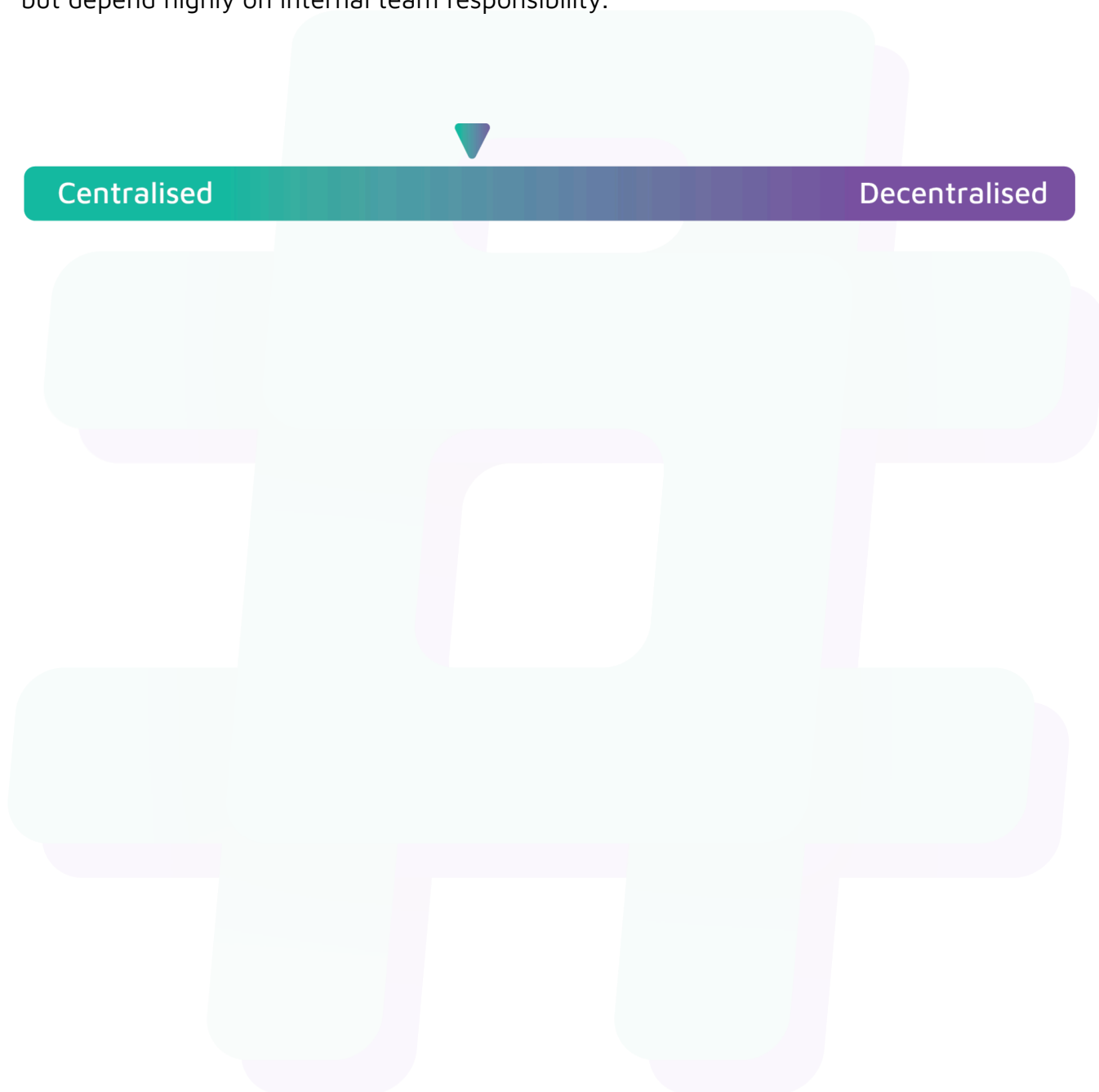
Acknowledged



## Centralisation

The Line Next project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



## Conclusion

After Hashlock's analysis, the Line Next project seems to have a sound and well-tested code base, however, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

## Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

### **Manual Code Review:**

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular Line Next of investigation.

### **Vulnerability Analysis:**

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

**Documenting Results:**

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

**Suggested Solutions:**

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

# Disclaimers

## Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

## Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

## About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamLine Nextd adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

**Website:** [hashlock.com.au](https://hashlock.com.au)

**Contact:** [info@hashlock.com.au](mailto:info@hashlock.com.au)





**#hashlock.**